

# Java EE

---

JAVA BEANS & ENTREPRISE JAVA BEANS

# Plan

---

- Java Beans :
  - Présentation
  - Événements
  - Propriétés
  - Introspection
  - Manipuler et déployer les JavaBeans
- Entreprise Java Beans :
  - Présentation
  - EJB session
  - EJB entité
  - Outils pour développer et mettre en œuvre des EJB
  - Déploiement des EJB
  - Appel d'un EJB par un client
  - EJB orientés messages

# Java Bean

---

# JB : Objectifs

---

- Définir un modèle de composant logiciel pour Java
  - Blocs de construction pour composer des applications
  - JavaBean en tant qu'applications
- Des tierces parties peuvent créer des composants Java qui seront assemblés ensemble pour créer des applications pour les usagers finaux
- Architecture neutre de composants
- Simplicité

# JB : Définition

---

- Un Java Bean est un composant logiciel réutilisable qui peut être manipulé visuellement dans un outil d'assemblage (éditeur graphique, BeanBox...).
- Exemples d'outils d'assemblage
  - Editeur de page Web
  - Editeur visuel d'applications
  - Constructeur de GUI
  - Editeur d'applications serveurs

# JB : Caractéristiques distinctives

---

- Support pour l'introspection de telle sorte qu'un outil d'assemblage puisse analyser comment le bean travaille
- Support pour la “customization” de telle sorte que lorsqu'il utilise un constructeur d'applications, l'utilisateur puisse configurer l'apparence et le comportement du bean
- Support pour les événements en tant que métaphore simple de communication qui puisse être utilisée pour connecter les beans entre eux.
- Support pour les propriétés à des fins de customization et de programmation.
- Support pour la persistance, de telle sorte qu'un bean puisse être configuré par un éditeur d'applications et que son état configuré puisse être sauvé et récupéré plus tard

# JB : Principales caractéristiques

---

- Ensemble de propriétés exposées
  - attributs nommés (variables d'instance)
- Ensemble de méthodes que les autres composants sont autorisés à invoquer
  - par défaut, toutes les méthodes publiques
- Ensemble d'événements déclenchés
  - une façon d'informer les autres composants que quelque chose d'intéressant est survenu

# JB : Lien à un état distant (remote state)

---

- Activation locale
  - le modèle d'exécution de base est qu'un bean s'exécute dans le même espace d'adresse que son contenant
- Mécanismes principaux d'accès au réseau
  - Java RMI
  - Java IDL: modèle d'objet distribué de CORBA (OMG)
  - JDBC: Java Database API
  - Migration d'objets sur le réseau

# JB : Utilisation

---

- Les éditeurs d'applications offrent souvent des scénarios de construction très différents.

# JB : Persistence I

---

- Tous les beans doivent implémenter Serializable ou Externalizable
  - le mécanisme de sérialisation automatique de Java
  - le mécanisme d’“externalization” dans un flot (stream)
    - contrôle total sur le format des données,
    - permet de mimer des formats de données existants.
- un bean ne doit pas normalement stocker des pointeurs sur des beans externes
  - utiliser les variables “transient” pour pointer vers les autres beans ou les “listeners” d’événements
  - la responsabilité du beans ‘conteneur’ est de garder la trace des liens inter-beans, de les emmagasiner et de les reconstruire lorsque nécessaire.

# JB : Persistence II

---

- Le designer d'un bean peut spécifier qu'un bean ne peut pas être restauré simplement en le sauvant et en restaurant seulement ses propriétés.
  - Utiliser le mécanisme attribut/valeur du FeatureDescriptor
    - l'attribut "hidden-state" = Boolean true.
    - Il faut utiliser la sérialization ou l'externalization pour sauver et / ou restaurer l'état du bean

# JB : Événements

---

- Les événements permettent de propager et de notifier les changements d'états entre
  - un objet source
- et
  - un ou plusieurs objets listener.

# JB : Objectifs des événements

---

- Capture et déclenchement des événements dans des environnements de scripts.
- Découverte des événements qu'une classe donnée d'objets peut générer.
- Découverte des événements qu'une classe donnée d'objets peut observer.
- Mécanisme d'enregistrement commun qui permet la manipulation dynamique des relations entre les sources d'événements et les listeners.

# JB : Implémentation

---

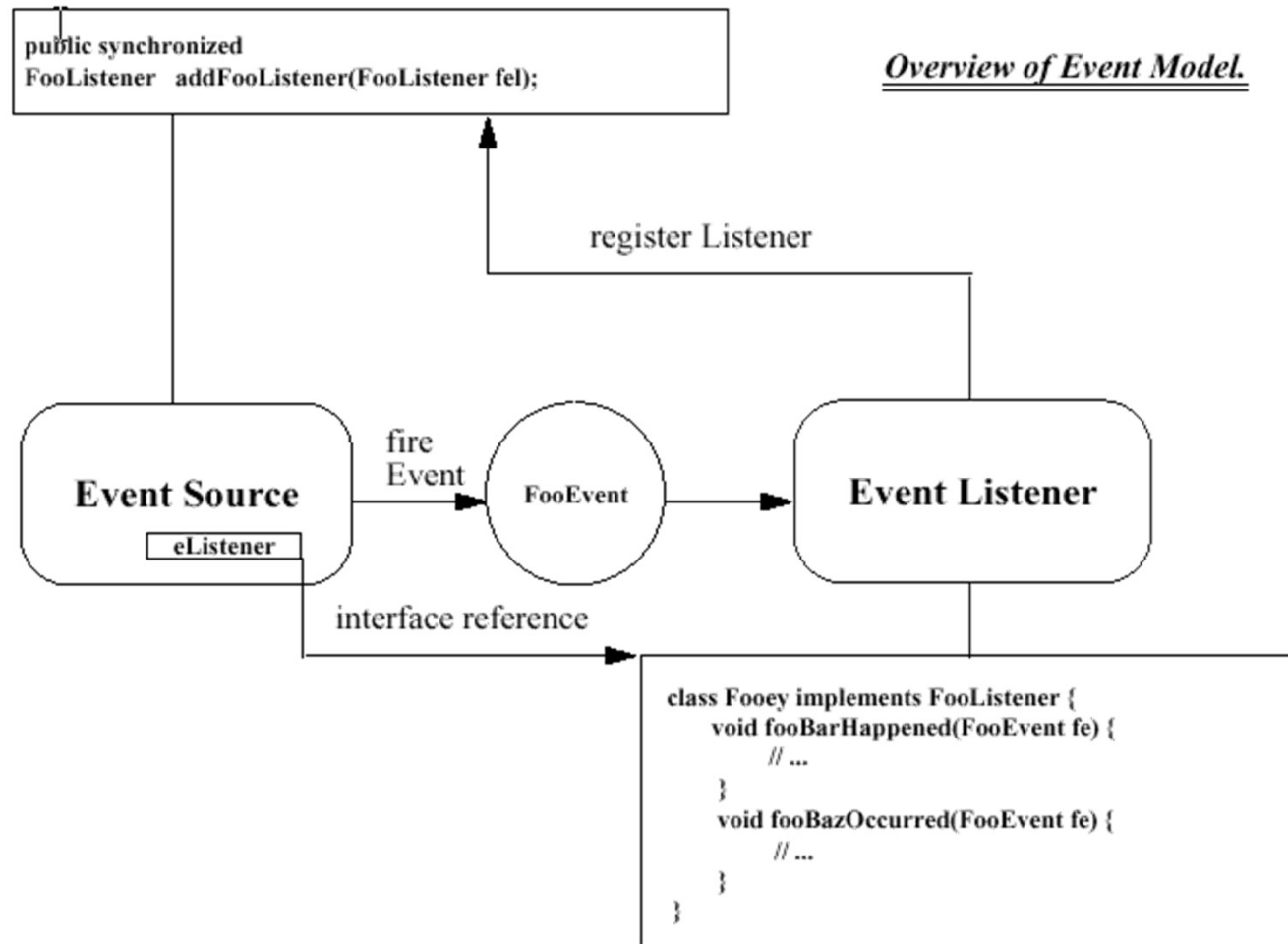
- Notifications d'événements
  - propagées des sources vers les listeners par invocation de méthodes Java
- `java.util.EventListener`
  - Groupe d'événements de la même catégorie
  - A chaque type d'événement correspond une méthode distincte pour la notification.
- Classe de listeners d'événements
  - implémente un ensemble donné d'interfaces `EventListener`.
- `java.util.EventObject`
  - l'état associé avec la notification d'un événement

# JB : Implémentation

---

- Sources d'événements
  - méthodes d'enregistrement doivent se conformer à un design pattern spécifique de noms
- classes d'adapteurs sur mesure
  - peuvent s'interposer entre la source et un ou plusieurs listeners pour les relier ou ajouter au comportement de base des listeners.

# JB : Notification d'événements



# JB : Objets, état de l'événement

---

- Par convention, les classes qui décrivent l'état d'un événement ont des noms qui se terminent par "Event".
- Des sous-classes de `java.util.EventObject` peuvent être créées
  - afin de permettre des distinctions logiques entre les types d'événements,
  - même si elles partagent les mêmes données.

# JB : Interfaces Event Listeners

---

- Pour définir et regrouper les méthodes de gestion des événements.
- Les interfaces EventListener héritent de `java.util.EventListener`
- Par convention, le nom de ces interfaces se termine en “Listener”.

# JB : Design pattern associé aux événements

---

- un design pattern standard pour
  - la documentation du système d'événements
  - pour déterminer par programmation les interfaces des listeners réalisées par des tiers
  - construction automatique d'adapteurs génériques.
- La signature de ce design pattern est:
- `void <eventOccurenceMethodName>(<EventStateObjectType> evt);`
- où `<EventStateObjectType>` est une sous-classe de `java.util.EventObject`.

# JB : Notes sur les éditeurs d'applications

---

- Les méthodes de gestion des événements devraient n'avoir que
  - un seul argument
    - instance de la sous-classe `java.util.EventObject`.
- Les éditeurs d'applications
  - devraient aussi supporter des signatures plus libres
  - ne devraient pas discriminer entre
    - les méthodes qui se conforment strictement au design pattern
    - et celles qui ne s'y conforment pas strictement.

# JB : Enregistrement d'un Event Listener

---

- Les classes qui déclenchent des événements doivent fournir des méthodes
  - d'enregistrement des listeners (add)
  - de "dés-enregistrement" des listeners (remove)
- Format:

```
synchronized    public void add< ListenerType>(< ListenerType> listener);
```

```
synchronized    public    void    remove<ListenerType>(<ListenerType>  
listener);
```

# Example

---

```
public interface ModelChangeListener extends java.util.EventListener {
    void modelChanged(EventObject e);
}

public abstract class Model {
    private Vector listeners = new Vector(); // list of Listeners

    public synchronized void
    addModelChangeListener(ModelChangeListener mcl) {
        listeners.addElement(mcl);
    }

    public synchronized void
    removeModelChangeListener(ModelChangeListener mcl) {
        listeners.removeElement(mcl);
    }
}
```

# synchronized

---

- Les segments de code qui accède le même objet à partir de threads distincts parallèles sont appelés sections critiques.
- En Java, une section critique
  - peut être un bloc ou une méthode
  - identifiée par le mot-clé synchronized.
  - Un verrou ('lock') est associé avec tout objet qui contient du code synchronisé

# Synchronized : verrouillage

---

- Lorsque le contrôle entre dans une méthode synchronisée,
  - le thread qui a appelé cette méthode
    - verrouille l'objet dont la méthode a été appelé.
- Les autres threads ne peuvent pas appeler une méthode synchronisée du même objet
  - jusqu'à ce que l'objet soit déverrouillé.
- L'acquisition et la libération d'un verrou est faite automatiquement et atomiquement par l'exécutable Java.

# JB : Propriétés

---

- Les propriétés peuvent être visibles dans les environnements de scripts
- Les propriétés peuvent être accédées par programmation par les autres composants en invoquant les accesseurs (getter and setter methods)
- Feuille de propriétés → fait partie du processus de personnalisation (customization) d'un composant
- En général, les propriétés sont persistentes

# JB : Méthodes d'accès

---

- Les propriétés sont toujours accédées par invocation de méthodes sur les instances qui les contiennent.
- Noms
  - les méthodes d'accès peuvent avoir des noms arbitraires.
  - Par convention
- `PropertyType getFoo(); // simple getter`
- `void setFoo(PropertyType value); // simple setter`
- même si dans un éditeur de scripts permet d'écrire quelque chose du genre
- `"b.Label = foo"`
- il y aura invocation d'une méthode sur l'objet cible

# JB : Propriétés indexées

---

- Pour spécifier un indice qui identifie quelle valeur obtenir
- Les indices de propriétés doivent être de type int (Java).
  - Cette contrainte sera relaxée dans le futur pour permettre d'autres types d'indice.
  - `PropertyType getter(int index); // indexed getter`
  - `PropertyType[] getter(); // array getter`
  - 
  - `void setter(int index, PropertyType value); // indexed setter`
  - `void setter(PropertyType values[]); // array setter`
- peut déclencher une exception
- `java.lang.ArrayIndexOutOfBoundsException`

# JB : Exceptions et méthodes d'accès

---

- Les méthodes d'accès simples et indexées peuvent déclencher des exceptions.

# JB : Propriétés liées (bound)

---

- S'il existe un service de notification des changement d'une propriété  
→ bound
- PropertyChangeListener event listener
  - pour informer des mises-à-jour des propriétés liées simples.

```
public void addPropertyChangeListener      (PropertyChangeListener x);  
public void removePropertyChangeListener (PropertyChangeListener x);
```

- invocation de la méthode suivante pour chacun des
  - aListener.propertyChange(PropertyChangeEvent evt)

# JB : Propriétés liées (bound)

---

- La source de l'événement devrait déclencher l'événement après la mise-à-jour de son état interne.
- Classe utilitaire `PropertyChangeSupport`
  - pour gérer la liste des `PropertyChangeListeners`
  - pour déclencher les événements `PropertyChange`.

# JB : Propriétés contraintes

---

- Les propriétés sont dites contraintes lorsque d'autres beans peuvent
  - souhaiter valider le changement
  - rejeter un changement s'il est inapproprié.
- `public PropertyType getFoo();`
- `public void setFoo(PropertyType value) throws PropertyVetoException;`

# JB : Propriétés contraintes

---

- Si le récepteur de l'événement ne souhaite pas que la modification soit faite
  - → il déclenche une exception `PropertyVetoException`
- la responsabilité de la source de
  - capturer cette exception
  - rétablir l'ancienne valeur
  - rapporte le retour l'ancienne valeur via un nouvel événement `VetoableChangeListener.vetoableChange`.
- La source devrait déclencher ce type d'événement avant de mettre à jour son état.

# JB : Propriétés contraintes

---

- Il existe une classe utilitaire `VetoableChangeSupport`
  - pour gérer la liste des `VetoableChangeListeners`
  - pour déclencher les événements `VetoableChange`
  - pour capturer les exceptions `PropertyVetoExceptions` et émettre les événements de réversion nécessaires.

# JB : Ecouter les propriétés à la fois liées et contraintes

---

- Si
  - un bean supporte une propriété qui est à la fois liée et contrainte
- alors
- elle devrait déclencher
  - un événement `VetoableChangeListener.vetoableChange` avant la mise-à-jour de la propriété
  - un événement `PropertyChangeListener.propertyChange` après la mise-à-jour de la propriété.

# JB : Vérification des événements sans effet

---

- Si
  - une méthode setter est invoquée avec un argument égal à la valeur courante de la propriété
- alors
  - il est recommandé que le bean ne déclenche pas d'événement de type
  - `VetoableChangeListener.vetoableChange` ou
  - `PropertyChangeListener.propertyChange`

# JB : Support optionnel pour les propriétés liées

---

- `void addPropertyChangeListener`
  - `(String propertyName, PropertyChangeListener listener);`
- `void removePropertyChangeListener`
  - `(String propertyName, PropertyChangeListener listener);`
- le bean doit aussi supporter le design pattern standard pour l'enregistrement des event listeners, qui ne prend qu'un seul argument.
  - alternative
- `void add<PropertyName>Listener(PropertyChangeListener p);`
- `void remove<PropertyName>Listener(PropertyChangeListener p);`

# JB : Support optionel support pour les propriétés contraintes nommées

---

- `void addVetoableChangeListener`
  - `(String propertyName, VetoableChangeListener listener);`
- `void removeVetoableChangeListener`
  - `(String propertyName, VetoableChangeListener listener);`
- le bean doit aussi support le design pattern standard pour l'enregistrement des event listeners, qui ne prend qu'un seul argument.
  - alternative
- `void add<PropertyName>Listener(VetoableChangeListener p);`
- `void remove<PropertyName>Listener(VetoableChangeListener p);`

# JB : Classes de support en Java

---

- class `java.beans.PropertyChangeEvent`
  - `null` peut être utilisé comme ancienne et/ou nouvelle valeur si la valeur réelle n'est pas connue.
  - `null` peut être utilisé comme nom de la propriété modifiée pour indiquer qu'un ensemble arbitraire de propriétés ont été modifiées.
- interface `java.beans.PropertyChangeListener`
- class `java.beans.PropertyChangeSupport`
  - Aucun événement n'est déclenché si les anciennes valeurs sont égales et non-nulles.
- class `java.beans.PropertyVetoException`
- interface `java.beans.VetoableChangeListener`
- class `java.beans.VetoableChangeSupport`

# JB : Introspection

---

- Pour découvrir
  - les propriétés
  - les événements
  - les méthodes
- Pour éviter l'usage d'un langage distinct de spécification pour définir le comportement d'un Java Bean.

# JB : Comment fonctionne l'introspection?

---

- Par défaut
  - Un mécanisme réflexif de bas-niveau
  - Utilisation de design patterns simples pour déduire à partir des méthodes quels sont les propriétés, les événements et les méthodes publiques qui sont disponibles.
    - design patterns = noms et signatures stéréotypés pour un ensemble de méthodes et/ou d'interfaces
- L'interface BeanInfo
  - pour exercer un contrôle complet et précis sur les propriétés, les événements et les méthodes qui sont exposées.

# JB : Réflexivité: Design patterns

---

- Propriétés
  - simple → get and set
  - boolean → is and set
  - indexées → get, set, get\_\_(int x) and set\_\_(int x, \_\_)
  -
- Événements
  - Multicast → add/remove\_\_Listener
  - Unicast → add\_\_Listener(\_\_)
  - throws  
java.util TooManyListenersException;
- Méthodes
  - toutes les méthodes publiques sont exposées

# JB : Spécification explicite

---

- L'interface BeanInfo
  - les méthodes permettent de connaître
    - événements
    - propriétés
    - méthodes
    - information globale.
- <BeanClassName>BeanInfo
  - La classe qui sert de BeanInfo peut choisir de ne spécifier qu'une partie de l'information sur le comportement du bean.
- Les éditeurs d'applications devraient toujours utiliser l'interface de l'introspecteur pour combiner l'information provenant de plusieurs sources d'informations pour construire le descripteur (bean info) définitif d'un bean donné.

# JB : Analyse d'un bean

---

- La classe `java.beans.Introspector`
  - traverse la hiérarchie des classes / superclasses d'une classe donnée.
  - A chaque niveau
    - Vérifie s'il existe une spécification explicite (`BeanInfo`)
    - Si non, effectue une analyse implicite fondée sur les design patterns
- Si
- une classe fournit un `BeanInfo` explicite sur elle-même
- alors
  - l'information fournie par le `BeanInfo` est ajoutée à l'information obtenue par l'analyse des sous-classes
  - cette information explicite est considérée comme finale pour la classe courante et ses superclasses,
  - l'inspection s'arrête et ne va pas plus loin dans la hiérarchie des superclasses

# JB : Package java.beans.\*; //Bean Info

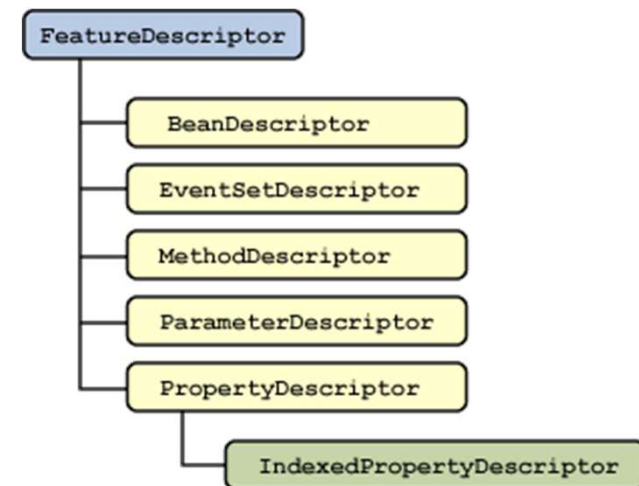
---

- interface
  - BeanInfo
    - public BeanInfo[] getAdditionalBeanInfo()
      - rend une collection arbitraire d'objets de type BeanInfo
      - cette collection fournit des informations additionnelles sur le bean courant.
- class
  - SimpleBeanInfo
    - applique l'introspection de bas niveau et les design patterns pour l'analyse automatique du bean.

# JB : Package java.beans.\*; // descripteurs

---

- FeatureDescriptor
  - displayName, shortDescription...
  - BeanDescriptor
    - classe du bean, customizer
  - EventSetDescriptor
    - Groupe des événements déclenchés sous forme d'invocation de méthodes sur une seule interface d'event listener
  - PropertyDescriptor
    - public void setPropertyEditorClass(Class propertyEditorClass)
    - IndexedPropertyDescriptor
  - MethodDescriptor
  - ParameterDescriptor



# JB : Exemple d'introspection

---

```
BeanInfo info =Introspector.getBeanInfo(SimpleBean.class);  
for (PropertyDescriptor pd : info.getPropertyDescriptors()  
    )  
    System.out.println( pd.getName() );
```

# JB : Package java.beans.\*; //introspection

---

- IntrospectionException
- Introspector
  - Recherche des BeanInfo
    - via le nom de la classe complet (incluant le nom du package) + “BeanInfo”
    - via le seul nom de la classe + “BeanInfo”
      - recherche dans le chemin de recherche par défaut
        - Dépendant de l’implémewntation
      - public static void setBeanInfoSearchPath(String path[])
    - Réfexivité au bas niveau
      - If a feature's getter (for example, getMethodDescriptor()) method returns null, low-level reflection is then used for that feature.

# JB : Personnalisation (Customization)

---

- Personnaliser l'aspect et le comportement d'un JavaBean
  - Feuille de propriétés
    - il y a un éditeur de propriétés pour chaque propriété
  - Customizer
    - contrôle la personnalisation du bean
      - par exemple, un wizard

# JB : Editeur de propriétés

---

- une classe
    - qui fournit un certain nombre de façons de
      - lire
      - écrire
    - qui sera instanciée
      - en tant que partie d'une feuille de propriétés
      - en tant que champ d'un composant customizer plus complexe
- } la valeur de la propriété

# JB : Déterminer les éditeurs de propriétés

---

- `Java.beans.PropertyEditorManager`
  - Registre d'éditeurs de propriétés
    - établit une correspondance entre les types Java et les classes d'éditeurs de propriétés.
    - Default PropertyEditors
      - Java primitive types "boolean", "byte", "short", "int", "long", "float", and "double";
      - les classes `java.lang.String`, `java.awt.Color`, `java.awt.Font`.
  - Pré-chargé avec les éditeurs pour les types de base de Java
  - Recherche d'un éditeur
    - éditeur de propriétés spécifiquement chargé
      - `BeanInfo`, (`propDescriptor.setPropertyEditorClass(MoleculeNameEditor.class)`)
      - `PropertyEditorManager`
    - nom complet (incluant le nom du package) + "Editor"
    - chemin de recherche des packages
      - `public static synchronized String[] getEditorSearchPath()`
      - Par défaut pour le bean box, `sun.beans.editors`

# Enterprise Java Bean

---

# EJB: Gamme de services implicites

---

- Gestion du cycle de vie
  - Gestion de l'état
  - Sécurité
  - Transactions
  - Persistance
  - Localisation des composants transparente
  - (comparable à objets distribués CORBA)
  - Répartition de charge, pooling
- Le développeur se focalise sur les aspects métier

# EJB 2.0

Lifecycle interface  
- create  
- remove  
- find  
Remote(rmi) ou  
Local  
Implementé par  
le conteneur

Implementation du composant:  
-logique métier (code)  
-callbacks pour le conteneur  
(ejbCreate, ejbPassivate,  
ejbLoad, ...)

Descripteur de déploiement  
-comportement transactionnel (Tx  
attributes)  
-Sécurité (ACLs)  
-Persistance (entity beans)  
-Ressources (DataSources ...)

DD  
`<resource-ref>`  
`<..name>jdbc/MyDS`  
...  
`<trans-attribute>`**Required**

Client

Home

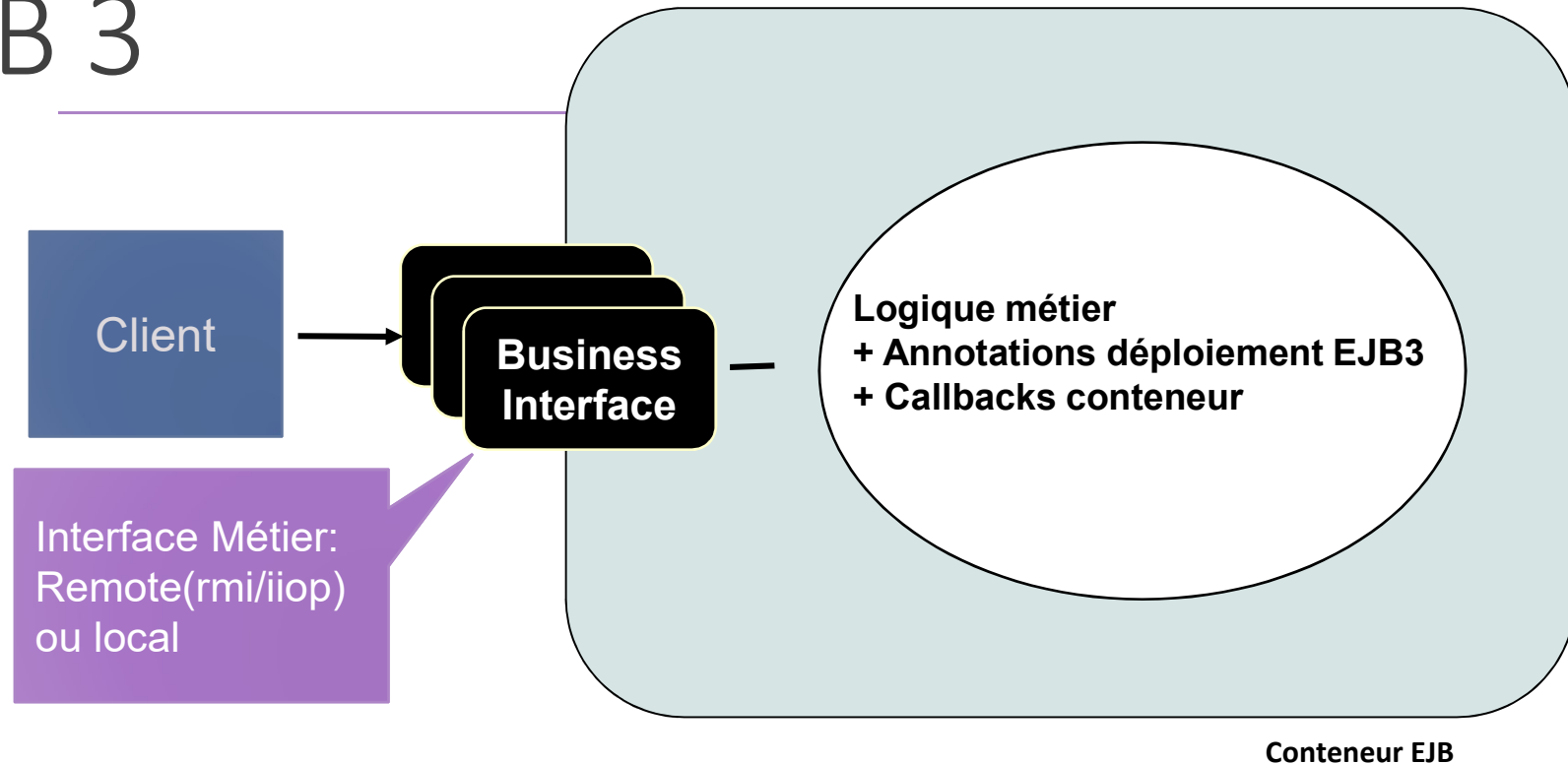
**Logique métier ... +  
Callbacks ... +**  
ic=new InitialContext();  
ds = (DataSource) ic.lookup(  
"java:comp/env/jdbc/MyDS");

Composant

Conteneur EJB

Interface Métier:  
Remote(rmi) ou  
local

# EJB 3



Simplification : 1 classe, 1 ou 2 interfaces

- L'interface Home (cycle de vie) disparaît

Le descripteur de déploiement devient facultatif

- Remplacé par des annotations java dans le bean

- Si présent tout de même, priorité au DD sur les annotations

# EJB: Interface métier

---

- Remote (RMI / IIOP) ou Local
- Vue client de l'EJB
- Declare les méthodes métier
- Implementée par les outils intégrés à la plateforme EJB - au moment du déploiement

# Exemple : interface métier

---

```
package facturation;
```

```
public interface Facturation {  
    void init( );  
    void creerFacture(String numfact, double  
montant);
```

```
    Facture getFacture(String numfact);
```

```
    // ...
```

```
}
```

# EJB: Implémentation du Bean

---

- Implémente les méthodes de l'interface métier
- Peut hériter d'un autre EJB, ou d'un POJO
- Spécifie ses caractéristiques de déploiement par annotations
  - Type de bean
  - Comportement : transactions, sécurité, persistance...
  - Callbacks conteneur

# Exemple : Implém. de Bean

---

```
package facturation;
```

```
@Stateful
@Remote(Facturation.class)
public class FacturationBean implements Facturation {

    void init( ) {
        // ...
    }

    Facture getFacture(String numfact) {
        // ...
    }
    // ...
}
```

# EJB: Code client

---

```
Context ctx = new InitialContext();
```

```
// appel JNDI pour obtenir une référence  
//à l'interface
```

```
Facturation fact =  
    (Facturation) ctx.lookup(  
        "facturation.Facturation");
```

```
// appel méthode métier  
Facture f = fact.getFacture(numfact);
```

# EJB: Entité

---

- Représente des données dans la base de données
- Container-Managed Persistence (CMP) or Bean-Managed Persistence (BMP)
  - En mode CMP, le conteneur EJB gère la persistance du bean (pas d'accès BD dans le code).
  - En mode BMP, c'est le développeur du Bean qui gère la persistance (par exemple, JDBC, JDO ...).

# EJB: Bean Session

---

- Gestion des interactions entre beans entité ou session, accès aux ressources, réalisation d'actions sur demande du client
  - Objets métier non persistants
  - Stateful ou Stateless - maintient ou pas un état interne en mémoire
- => Un Bean Stateful encapsule la logique métier et l'état spécifiques à un client

# EJB : Message Driven Bean

---

- Composant asynchrone
- Execution sur réception d'un message JMS
  - Méthode onMessage()
  - Appels à d'autres beans, etc...
- Descripteur de déploiement
  - Associations Bean / ressource JMS
  - Ressources JMS (ex. Queue ou Topic)

eclipse2020-ws-CI1-23 - PremiersPas/src/main/java/model/TestJBean.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

```
8 public class TestJBean {
9
10     public static void main(String[] args) {
11         //Création d'une instance du JB
12         JBean monBean = new JBean();
13
14         //Test de la propriété simple
15         System.out.println("Valeur de la propriété simple = " + monBean.getProprieteSimple());
16         monBean.setProprieteSimple(10);
17         System.out.println("Valeur de la propriété simple = " + monBean.getProprieteSimple());
18
19         //Test de la propriété booleene
20         System.out.println("Valeur de la propriété booleene = " + monBean.isProprieteBooleene());
21         monBean.setProprieteBooleene(false);
```

Console

<terminated> TestJBean [Java Application] C:\Program Files\Java\jdk-13.0.1\bin\javaw.exe (16 févr. 2023 à 21:59:39 – 21:59:39)

```
Valeur de la propriété simple = 1
Valeur de la propriété simple = 10
Valeur de la propriété booleene = true
Valeur de la propriété booleene = false
Contenu de la propriété indexee :
0      0      0      0      0      0      0      0      0
Contenu de la propriété indexee :
0      10     20     30     40     50     60     70     80     90
Contenu de la propriété indexee :
0      10     20     30     40     50     60     70     80     90
Contenu de la propriété indexee :
0      20     40     60     80     100    120    140    160    180
Valeur de la propriété liée = 2
```

Writable Smart Insert 8:23:210

# Tutoriel 4

## Java Bean